How Blockchain Anchoring Works in ELIXOR

Fingerprints, the MedicalAudit smart contract, the Hardhat log, and Verify integrity, explained step by step

Project: ELIXOR (github.com/Zageraus/Elixo) · Network: local Hardhat chain, chain ID 31337 · 27 October 2026

In one paragraph. Every medical report and prescription in ELIXOR gets a SHA-256 *fingerprint*: a 64-character code that changes completely if even one byte of the record changes. The backend writes that fingerprint to a smart contract on a blockchain, where nobody, not even the database owner, can edit or delete it. Later, the **Verify integrity** button re-computes the fingerprint from the record as it is now and compares it with the one on the blockchain. If they match, the record is provably unchanged; if not, it was tampered with. The medical data itself never goes on the blockchain.

1. Why a blockchain, and why only a fingerprint

A database is controlled by whoever runs it. Someone with access could quietly change a dosage or swap a lab report, and the audit log could be edited too. A blockchain is a shared, append-only record: once a transaction is in a block, it can't be changed without rewriting every block after it, which the network rejects.

Storing whole medical files on a blockchain would be expensive and a privacy problem, because anything on a public chain is permanent and visible to everyone. So ELIXOR stores only a **32-byte fingerprint** per record. The fingerprint proves what the record contained without revealing any of it.

Property of SHA-256	Why it matters here
Same input, same output	The same file always gives the same fingerprint, so it can be checked later
Tiny change, completely different output	Changing one character of a report gives an unrelated fingerprint, so any edit is visible
One-way	Nobody can rebuild the report from its fingerprint, so nothing private is exposed
Collisions practically impossible	Nobody can craft a different file with the same fingerprint

2. The parts involved

Part	Where it lives	Job
MedicalAudit smart contract	`contracts/contracts/MedicalAudit.sol`, deployed at `0x5fbd…0aa3`	Stores fingerprints; refuses overwrites; answers "what is stored for this record?"
Local Hardhat chain	`npm --prefix contracts run node`, port 8545	A private Ethereum network on your PC that mines each transaction instantly
Backend signer wallet	Key in `backend/.env`; address `0xf39f…2266`	Signs and pays for the anchoring transactions (free test ETH)
Blockchain service	`backend/src/services/blockchain.service.ts`	Computes fingerprints, sends transactions, reads records back
`blockchain_transactions` table	Supabase PostgreSQL	Remembers which transaction anchored which record, its block and status
Verify integrity button	`frontend/src/components/VerifyIntegrity.tsx`	Asks the backend to re-check a record and shows the result
Setup and backfill scripts	`contracts/scripts/setup-local.ts`, `backend/src/scripts/anchor-backfill.ts`	Deploy the contract; anchor all existing records

3. The smart contract

The contract is small. For each record it stores four things, and it exposes three functions; the backend uses `anchorRecord` to write and `getRecord` to read:

```
struct AuditRecord {
    bytes32 dataHash;     // the SHA-256 fingerprint
    address submittedBy;  // which wallet anchored it
    uint256 timestamp;    // block time when it was anchored
    string  recordType;   // "report" or "prescription"
}
mapping(bytes32 => AuditRecord) public records;   // record ID -> its audit record

function anchorRecord(bytes32 recordId, bytes32 dataHash, string recordType)  // write once
function verifyRecord(bytes32 recordId, bytes32 dataHash) returns (bool)      // does it match?
function getRecord(bytes32 recordId) returns (AuditRecord)                    // read it back
```

- **Write once:** `anchorRecord` fails with "Record already anchored" if the record ID already has an entry, so a fingerprint can never be replaced.
- **Only authorised writers:** only the contract's owner and wallets the owner authorised can anchor; anyone else gets "Not authorized".
- **Reading is free:** `getRecord` and `verifyRecord` only read, so they cost no gas and create no block.

- **Tested:** 6 Hardhat unit tests cover anchoring and verifying, duplicates, unauthorised callers and adding signers. All pass.

---

4. What exactly gets fingerprinted

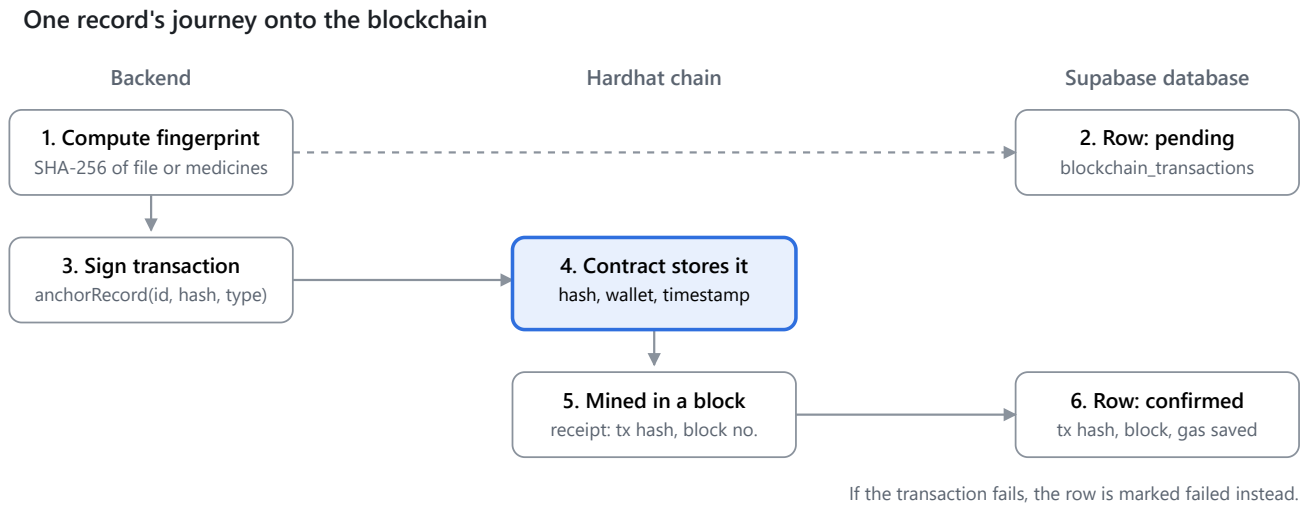
Record	Fingerprint (dataHash)	Record ID on the chain
Report	SHA-256 of the uploaded file's bytes, computed at upload and also saved in `medical_reports.file_hash`	The record's database UUID, hashed with keccak-256 into 32 bytes: `ethers.id(uuid)`. The same UUID always gives the same ID, so the record can be found again later.
Prescription	SHA-256 of its medicines written as `[[name, dosage, frequency, duration_days], …]` in that fixed order	

The fixed order for prescriptions matters. PostgreSQL's JSONB type doesn't keep the order of keys, so hashing the stored JSON directly could give a different fingerprint for the same prescription. Listing the four values in a fixed order makes the fingerprint stable.

---

5. How a record gets anchored

This happens automatically when a report is uploaded or a prescription is issued (while `BLOCKCHAIN_ENABLED=true`), and for all existing records when the backfill script runs.



Steps 3 to 5 take a fraction of a second on the local chain; on the Sepolia test network they take about 12 to 30 seconds.

1. **Compute the fingerprint** of the file or the medicines, as in section 4.
2. **Record the attempt.** A row is added to `blockchain_transactions` with status `pending`. Earlier unconfirmed attempts for the same record are removed first, so retries don't pile up.
3. **Check the chain** is reachable and the contract is deployed at the configured address, then **sign** a transaction calling `anchorRecord(recordId, dataHash, recordType)` with the backend's wallet.
4. **The contract checks and stores.** It refuses unknown wallets and already-anchored IDs; otherwise it saves the fingerprint, the wallet and the block time, and emits a `RecordAnchored` event.

5. **The transaction is mined** into a block and the backend receives a receipt with the transaction hash and block number.
6. **The row is confirmed** with the transaction hash, block number, gas used and time. The report or prescription card now shows **Anchored on Local Hardhat chain**.

---

6. Reading your Hardhat log

The log you captured is the terminal running the local chain. Every line is a request the backend or the setup script sent to it. It shows the contract being deployed, then 19 records anchored one by one, with no errors.

The deployment (block 1)

```
Contract deployment: MedicalAudit
Contract address:    0x5fbdb2315678afecb367f032d93f642f64180aa3
From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
Gas used:            598199 of 16777216
Block #1
```

- `setup:local` deployed the contract. `0xf39f…2266` is Hardhat's first built-in test account, which is also the backend's signer, so it's the contract's owner and already allowed to anchor.
- The address is the same every time on a fresh chain, because a contract's address is calculated from the deployer's address and its transaction count (0 here).
- Deploying cost 598,199 gas because it stores the contract's code on the chain.

The check-then-anchor cycle, repeated for each record

Log lines	What is happening
`eth_chainId`, `eth_getCode`	The backend confirms it's on the expected chain and that the contract exists at the address
`eth_call` → `MedicalAudit#getRecord`	The backfill asks whether this record is already anchored. A read only: no gas, no block
`eth_getTransactionCount`, `eth_estimateGas`, `eth_gasPrice`, `eth_maxPriorityFeePerGas`, `eth_getBlockByNumber`	Preparing the transaction: the wallet's next sequence number, the gas it will need, and the current fee
`eth_sendRawTransaction` → `MedicalAudit#anchorRecord`	**The anchoring itself:** the signed transaction is sent and the fingerprint is written to the contract
`Block #N`	Hardhat mines it immediately into its own block
`eth_getTransactionReceipt`	The backend waits for the confirmation, then saves the transaction hash in the database

What the gas numbers reveal

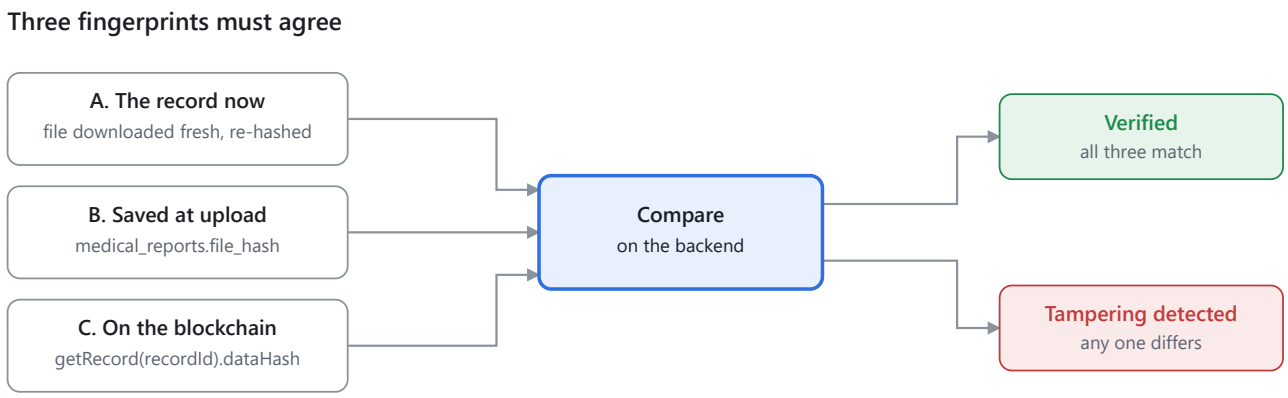
Blocks	Transactions	Gas each	What was anchored
2 to 11	10	117,257 (one 117,245)	Reports, `recordType = "report"`
12 to 20	9	117,329	Prescriptions, `recordType = "prescription"`

- **Why prescriptions cost 72 more:** transaction data costs 16 gas per non-zero byte and 4 per zero byte. "prescription" has 6 more letters than "report", turning 6 zero padding bytes into non-zero ones: 6 × (16 − 4) = 72 gas.
- **Why one report cost 12 less:** its random fingerprint happened to contain one more zero byte, which is 16 − 4 = 12 gas cheaper.
- **`Value: 0 ETH`:** no money is transferred; only gas is spent, and on the local chain that's free test ETH.
- **`Gas used: 117257 of 117257`:** the backend sets the gas limit to the exact estimate, and the transaction used all of it, as expected.

---

7. How Verify integrity works

The button appears on anchored reports and prescriptions. Clicking it asks the backend to check the record against two independent copies of its fingerprint.



For prescriptions, A is the current medicines re-hashed, and B is the hash recorded when it was anchored.

Comparison	Applies to	A mismatch means
A (file now) vs B (hash at upload)	Reports	The file was swapped or altered in storage
A (record now) vs C (contract)	Reports and prescriptions	The record changed after anchoring, even if someone also edited the hash saved in the database
A (medicines now) vs B (hash when anchored)	Prescriptions	A drug, dosage, frequency or duration was edited

Why C is the one that can't be faked: someone with database access could change both the file and the saved hash in B so they agree. They cannot change C, because the contract refuses to overwrite an anchored record. So any change after anchoring is caught by the A vs C comparison.

One detail that mattered: Supabase Storage briefly serves a cached copy of a file after it's overwritten, which hid a swapped file during testing. The check now downloads the file with a unique query string to bypass that cache.

The possible results

Shown on the card	When
Verified: unchanged since anchoring	All fingerprints match
Tampering detected	The file changed, the medicines changed, or the record no longer matches the contract
Not fully verifiable	The record isn't anchored on the current chain, anchoring is turned off, or the chain isn't running (the reason is shown)

Tested

- An untouched uploaded PDF and an untouched prescription both showed **Verified**.
- Replacing an uploaded file in storage was flagged as "file changed" and an on-chain mismatch.
- Editing a prescription's dosage directly in the database was flagged as an on-chain mismatch.
- Another patient asking to verify someone else's report was refused (404).

---

8. The local chain and how to restart it

The local Hardhat chain exists only in memory. Stopping it erases every block, so the contract and all fingerprints disappear, while everything in Supabase stays. After each restart:

```
npm --prefix contracts run node            # terminal 1: start the chain, keep it open
npm --prefix contracts run setup:local     # deploy the contract, update backend/.env
npm --prefix backend run dev               # restart the backend so it reads the new address
npm --prefix backend run anchor:backfill   # re-anchor every report and prescription
```

Until this is done, uploads still succeed, but their message says anchoring failed and explains why, and Verify reports that the chain can't be checked.

	Local Hardhat chain (today)	Sepolia test network (supported)
Cost	Free, instant	Free test ETH from a faucet
Permanence	Wiped on every restart	Permanent
Who can check	Only this PC	Anyone, on Etherscan
Speed per anchor	Under a second	About 12 to 30 seconds
Change needed	None	RPC URL, funded wallet key, `npm run deploy:sepolia`

9. What this proves, and what it doesn't

It proves

- A record is byte-for-byte the same as when it was anchored
- When it was anchored, and by which wallet
- That nobody replaced a fingerprint, because the contract can't overwrite one

It doesn't prove

- That the record was correct when it was uploaded
- Who in the hospital made a later change (the audit log covers that)
- Anything once the local chain restarts, until records are re-anchored. On a chain you run yourself, the proof is only as trustworthy as you; Sepolia removes that limit

10. Glossary

Term	Meaning
Blockchain	A chain of blocks, each holding transactions and linked to the previous block, so past entries can't be changed
Smart contract	A program stored on the blockchain; here, MedicalAudit
Transaction	A signed request that changes the blockchain, such as anchoring a record
Block	A batch of transactions; the local chain makes one block per transaction
Gas	The unit of work a transaction costs; paid in ETH (free test ETH here)
Wallet, address	A key pair that signs transactions; its public address, such as `0xf39f…2266`
Anchoring	Writing a record's fingerprint to the contract
SHA-256	The hash function that makes a record's 64-character fingerprint
keccak-256	Ethereum's hash function, used to turn a record's UUID into a 32-byte record ID
RPC call	A request to the chain, such as `eth_call` (read) or `eth_sendRawTransaction` (write); each line in the log is one
Hardhat	A development tool that runs a local Ethereum chain and tests contracts
Sepolia	A public Ethereum test network, free but permanent